ATML Project (VLMs and Counting Shapes) - Study 3

Final methodology and predictions (5 Nov 2026). Revised after Mohid's review and fixed before any main-run data.

> **Main hypothesis:** Unnamed clutter (squiggles, blobs, etc.) should hurt counting less than nameable clutter, because the model already ignores what it can't name. Not all clutter is equal -> a VLM is distracted more by things it can name than by things it can't name.
>
> **The confound we must separate:** most nameable clutter in our datasets (lookalikes, wrong colours, text about the counts) is also built to *look like a target*. So every test below keeps "has a name" and "looks like a target" apart.

General setup (same for every hypothesis)

- **Model:** Qwen3-VL-2B-Instruct (same model as the original paper and our studies 1 and 2), pinned version, 16-bit, greedy answers (no randomness), same question for every version of a scene.
- **Data:** Mohid's 4 datasets (Shapes, Shapes Hard, Objects, Lines) plus one new Shapes Hard version (below). Within a scene, the targets are pixel-identical across all versions -> all versions have the same correct answer, so any accuracy drop comes from the clutter.
- **Which images: all 500 scenes of every dataset** (train + validation + test). Nobody trains a model in this study, so all scenes can be used. The test split alone (75 scenes) gives about ±10 points of uncertainty on a paired difference, which is too wide. Any method trained later (e.g. disentanglement) trains on the train split and is evaluated on the test split only.
- **Answer start:** the model's reply is started with the first words of the required answer format ("circle:" for Shapes and Shapes Hard, "apple:" for Objects, "box:" for Lines), so it answers instead of explaining. The question itself is unchanged, the same start is used for every version of a dataset, and up to 64 new tokens are generated.
- **Accuracy:** counting datasets -> "exact" (all 4 counts right). Lines -> "acc" (right box).
- **Acc drop** = Acc on clean pictures - Acc on cluttered pictures, using the same scenes.
- **Per-image results:** the official scorer is run with `--out`, which saves per-image scores (including total_bias). Whenever we compare with clean, we subtract the same scene's clean result.
- **95% interval:** bootstrap over scenes (10,000 resamples). Regressions group images by scene in the same way.
- All predictions are written down before running the model. Every result gets reported, including ones that go against us.

New dataset version: nameable but unrelated shapes (Shapes Hard)

- Made by Mohid, without changing any existing image (the same way the squiggle versions were added).
- Nameable shapes that are **not** targets and don't look like targets: **heart, moon, arrow, cross**.
- Placed in **exactly the same spots**, with the same sizes and colours, as the squiggles. So the only difference from the squiggle versions is whether the clutter has a name.
- Called **named_unrelated** below (Mohid sets the final name).

	Looks like a target	Doesn't look like a target
Has a name	lookalikes, wrong_color (Shapes Hard); lookalikes (Objects)	**named_unrelated** (Shapes Hard); random_objects (Objects)
No name	not in our data (see limits)	squiggles_2 / _4 / _8 / _16 (Shapes Hard); squiggles, blobs (Shapes)

Stage 0: Checks before the main run

- **Floor check:** run a small trial (20 images per version) and check clean accuracy. If clean accuracy is below 0.30 (counting) or 0.40 (Lines), the 2B model is too weak for that dataset -> clutter can't make a near-zero score visibly worse.
- **If any dataset fails the floor check, every dataset is run with Qwen3-VL-8B**, so that no verdict mixes two models.
- **Parse check:** almost all answers can be read by the scorer (the "parsed" column).
- **Naming check for named_unrelated:** show each new shape on its own and ask "What shape is this? Answer in one word." Each must be named correctly (heart, moon, arrow, cross or a clear synonym). A shape the model can't name is reported and removed from the H1 analysis.

H1: Names vs looking like a target

Stage H1.1: Acc drop for every version

- Acc drop = Acc on clean pictures - Acc on cluttered pictures, same scenes. The bigger the Acc drop, the more that clutter hurt the VLM.

Stage H1.2: Names test (main) -> Shapes Hard

- Same spots, neither looks like a target, only naming differs:

```
Acc drop(named_unrelated) - avg( Acc drop(squiggles_2), Acc drop(squiggles_4),
      Acc drop(squiggles_8), Acc drop(squiggles_16) )
```

- Prediction: above 0 (nameable clutter hurts more).

Stage H1.3: Similarity test -> Shapes Hard and Objects

- Both have names, only "looks like a target" differs:

```
Shapes Hard: Acc drop(lookalikes) - Acc drop(named_unrelated)
Objects:     Acc drop(lookalikes) - Acc drop(random_objects)
```

Stage H1.4: Also reported (no verdict)

- Shapes: Acc drop(lookalikes) - avg(Acc drop(squiggles), Acc drop(blobs)). This mixes names and similarity, so it only shows their combined effect.

To evaluate: 95% interval for each difference -> the range in which the true value likely falls, given how noisy the results are.

- Whole range above 0 -> that factor hurts (names in H1.2, similarity in H1.3). Whole range below 0 -> the opposite. Range includes 0 -> no clear difference.
- H1.3 counts as supported only if both datasets agree.

Names test (H1.2)	Similarity test (H1.3)	Conclusion
above 0	above 0	Both names and similarity matter
above 0	not above 0	Names matter, similarity doesn't (strongest support for our idea)
not above 0	above 0	Similarity matters, names don't (our idea is not supported)
not above 0	not above 0	Neither explains the drops

H2: Do more complex squiggles hurt counting more, or less? -> Shapes Hard

When the model counts shapes, does a more complicated squiggle nearby distract it more, or does the model ignore it more because it looks like nothing it has a name for?

Idea	Reasoning	Prediction
"Busy clutter"	More curves make the picture busier, so the model gets more distracted.	Accuracy goes **down** as squiggles get more complex.
"Names"	The more complicated a squiggle is, the harder it is to name, so the model ignores it more.	Accuracy stays the **same or goes up** as squiggles get more complex.

Stage H2.1

- Run the model on all 4 squiggle versions (2, 4, 8, 16 curves) + the clean version and measure accuracy for each version. Acc = how often the model gets all 4 counts right.

Stage H2.2: Nameability score (does a real name exist?)

- Show every squiggle on its own to the model and ask 5 times: "*What does this shape look like? Answer in 1 word.*" Answers are sampled with temperature 0.7 so they can vary (fixed before the run).
- **Words that don't count as a name** (fixed before the run): squiggle, scribble, line, lines, curve, curves, wave, wavy, zigzag, doodle, loop, swirl, scrawl, stroke, mark, pattern, shape, abstract, random, design, drawing, sketch.
- **Nameability score = share of the 5 answers that name a concrete thing** (any answer not on the list above), from 0 (never names a thing) to 1 (always names a thing).
- Why not "same word every time": a model that answers "squiggle" 5 times out of 5 would look perfectly nameable, which is the opposite of what we mean.
- Each image gets the average nameability score of the squiggles in it.
- Sanity check: the named_unrelated shapes (heart, moon, ...) should score close to 1.

Stage H2.3: Two analyses

- **Main:** how accuracy changes each time the number of curves doubles (2 -> 4 -> 8 -> 16). This is the slope (each doubling = 1 equal step), fitted per image with images grouped by scene.
- **Untangling:** regression of "counted correctly?" on number of curves + nameability score together, images grouped by scene. This tells us, holding nameability fixed, whether more curves still change accuracy, and holding curves fixed, whether nameability still changes accuracy.

To evaluate:

- **Main slope:** whole range below 0 -> "busy clutter" idea wins. Whole range above 0 -> "names" idea wins. Range includes 0 -> unclear (possibly both effects cancel out).
- **Untangling:** nameability predicts accuracy and complexity doesn't -> names are what matter. Complexity predicts accuracy and nameability doesn't -> complexity is what matters.

Known limit: across the 4 versions, complexity and nameability change together, so the main slope alone can't separate them -> that's why Stage H2.2 measures nameability directly instead of assuming it. The score is measured with the same model, which is what we want (what the model can name), but it is not a human nameability rating.

H3: Every kind of clutter hurts counting -> all 3 counting datasets

- Acc drop for every cluttered version (including named_unrelated), compared with clean, on the same scenes.

To evaluate:

- Whole range above 0 -> that clutter hurts. Expected, but this is the baseline effect everything else builds on.
- The scorer also gives a McNemar p-value for each version vs clean -> report it next to the interval.

H4: The model counts clutter as real targets -> Shapes, Shapes Hard, Objects

- **Excess count** per image = total_bias (predicted total - true total, from the scorer's per-image output) minus the same scene's total_bias on the clean image. This removes over- or under-counting the model already does without clutter.
- Average the excess count per version.
- The "everything" version is left out of the comparison: it contains the lookalikes plus more distractors, so it would come out highest for a trivial reason. It is still reported.

To evaluate:

- Prediction: the excess count is highest for lookalikes among the single-trap versions in all 3 datasets.
- Excess count(lookalikes) - excess count(next-highest single-trap version): whole range above 0 = supported.

H5: The model easily counts incomplete shapes as complete shapes -> Shapes, Shapes Hard

Prediction: the VLM treats an incomplete shape (a gap in the outline, or one wavy side) as the complete shape it resembles. A square with a gap gets counted as a square.

Stage H5.1: What each incomplete shape imitates

- Every lookalike records which shape it imitates (lookalike_of) and its defect type (gap or wavy) in the "objects" column.
- Both questions explicitly say not to count broken or wavy shapes, so H5 uses both datasets.

Stage H5.2: Absorption rate (clean-subtracted)

- For each image and each shape type: extra = model's count on the lookalikes image - model's count on the same scene's clean image, kept between 0 and the number of incomplete shapes of that type.
- Absorption rate = total extra / total incomplete shapes -> the share of incomplete shapes that the model counted as complete ones (0 = never, 1 = always).

```
absorption rate = sum( extra ) / number of incomplete shapes
```

Stage H5.3: Compare with other clutter

- For the comparison versions, extra per distractor = (predicted total on that image - predicted total on the same scene's clean image) / number of distractors, kept at 0 or above.
- Shapes: compare with squiggles and blobs. Shapes Hard: compare with the squiggle versions and with named_unrelated.
- Difference = absorption rate(lookalikes) - extra per distractor(comparison clutter).

Stage H5.4: Also report

- Absorption rate split by defect type (gap vs wavy side) and by shape (circle, triangle, square, star).

To evaluate:

- Supported if: the difference in Stage H5.3 has its whole 95% range above 0 (incomplete shapes are counted more than other clutter) AND the absorption rate's whole range is above 0.5 (the model counts most incomplete shapes as complete -> "easily"). The 0.5 cut-off is fixed before the data.
- Difference above 0 but absorption rate not clearly above 0.5 -> partly supported (the model is fooled, but not most of the time).
- Difference range includes 0 -> incomplete shapes fool the model no more than other clutter (not supported).

H6: Text in the image fools the model -> Shapes Hard, Objects

Stage H6.1: Text vs the same amount of non-text clutter

- Any added item can hurt, so the text drop is compared with clutter that has the same number of non-text distractors:

```
Shapes Hard: Acc drop(text_traps) - avg( Acc drop(squiggles_2 ... squiggles_16) )
Objects:     Acc drop(text_traps) - Acc drop(random_objects)
```

Stage H6.2: Copy check (does it copy the written number?)

- Only texts that claim a count for a class (e.g. "4 stars", "cat: 4") are used. Bare numbers are not claims about a class, and about 1 in 4 of them equals some true count anyway.
- Copy rate = share of claims where the model's count for the claimed class equals the claimed number.
- Baseline = how often the model gives that same number for that class on the same scene's clean image.
- Difference = copy rate - baseline.

To evaluate:

- H6.1: whole range above 0 in both datasets -> text hurts more than the same amount of other clutter.
- H6.2: whole range above 0 -> the model reads the text and copies the claimed number.

H7: Tangled lines are harder to follow -> Lines dataset

Stage H7.1: Plain clutter

- Acc drop(crossings_0) (other lines present, but none crosses the target).

Stage H7.2: Entanglement

- **Main view:** only the "rank changed" images (where guessing "the line goes straight across" gives the wrong box). In about 70% of crossing images, and all of crossings_0, guessing straight across is right, so a model can score well without actually tracing the line.
- With all 500 scenes there are 149 / 157 / 187 rank-changed images at 2 / 4 / 8 crossings. These subsets are not the same scenes at every level, so the slope is fitted with a regression of "right box?" on number of crossings, with images grouped by scene.

Stage H7.3: Also report

- "jumped" (how often the model switched to the other line at a crossing) for each version.
- Accuracy for "rank kept" vs "rank changed" -> a big gap means the model is guessing, not tracing.

To evaluate: slope range below 0 -> more crossings = harder (supported).

H8: Colour gives a line an identity, like a name -> Lines dataset

```
Acc(crossings_4_colored) - Acc(crossings_4)
```

- Same lines, only the colour of the other lines differs (every other line gets its own colour; the target stays black).

To evaluate: whole range above 0 -> giving the other lines a separate colour helps the model keep track of the target. This links back to the main idea: an identity (a name or a colour) helps the model keep things apart.

Known limits (stated in the report)

- "Looks like a target but has no name" is not in our data, so that corner of the H1 table is untested.
- In Shapes Hard the squiggles are slightly larger than the lookalikes (mean radius 37 px vs 31 px). This works slightly against our hypothesis in comparisons between them.
- In Lines the target is always the black line.
- One model family (Qwen3-VL), answers generated as text and read by the scorer's parser.
- Nameability is measured with the model itself, not with human ratings.

Always reported (not used for verdicts)

Every version of every dataset (including "everything"), average count error (abs_err), parse rate, and how the model's answers are spread.

Changes after review (5 Nov 2026)

Made after Mohid's review of the first draft. At that point only a 20-images-per-version trial had been run, and none of its accuracy numbers had been looked at.

Change	Reason
H1 split into a names test (new named_unrelated vs squiggles) and a similarity test	All nameable clutter in the first draft also looked like the targets, so names and similarity could not be told apart
Nameability score = share of answers naming a concrete thing, with a fixed list of non-name words	"Same word every time" would rate "squiggle, squiggle, ..." as perfectly nameable
All 500 scenes instead of the test split; H7 slope by regression grouped by scene	75 test scenes give about ±10 points; H7's rank-changed subsets were tiny and unpaired
H4 and H5 subtract the same scene's clean result; "everything" left out of H4's comparison	overcnt includes over-counting the model already does on clean images
H6 compared with the same amount of non-text clutter; copy check only on class claims, against a clean baseline	Any added item can hurt; bare numbers often equal a true count by chance
8B everywhere if any dataset fails the floor check	Verdicts across datasets must not mix models
Reply started with the answer format ("box:", "apple:", "circle:"); answer limit raised from 40 to 64 tokens	Trial run: most Lines answers and some Objects answers started explaining and were cut off before giving a number, so the scorer could not read them